reply-bridge Plugin Tool

reply-bridge Plugin Tool

1. Course Content
   - Course Overview
   - Learning Objectives
3. reply-bridge Plugin Introduction
4. Case Demonstration
5. Source Code Analysis
   - 5.1 Plugin Entry and Hook Registration
   - 5.2 TTS Tag Extraction and Deduplication Mechanism
   - 5.3 TTS Server

1. Course Content

Course Overview

- The reply-bridge plugin is the speech synthesis bridge plugin of OpenClaw. It listens to OpenClaw's AI reply messages, parses the `<tts>...</tts>` voice tag from them, automatically calls the TTS tool on the MCP server for speech synthesis, and broadcasts through the USB audio device.
- Used together with the robot-control plugin, it enables voice dialogue interaction with the robot, letting the robot "speak".

Learning Objectives

1. Understand the features of the reply-bridge plugin
2. Master how to install, view, and perform basic management operations on plugins
3. Understand the working principle of the plugin calling TTS speech synthesis through the MCP protocol
4. Be able to use the `<tts>` tag to implement the voice broadcast feature in OpenClaw interaction scenarios

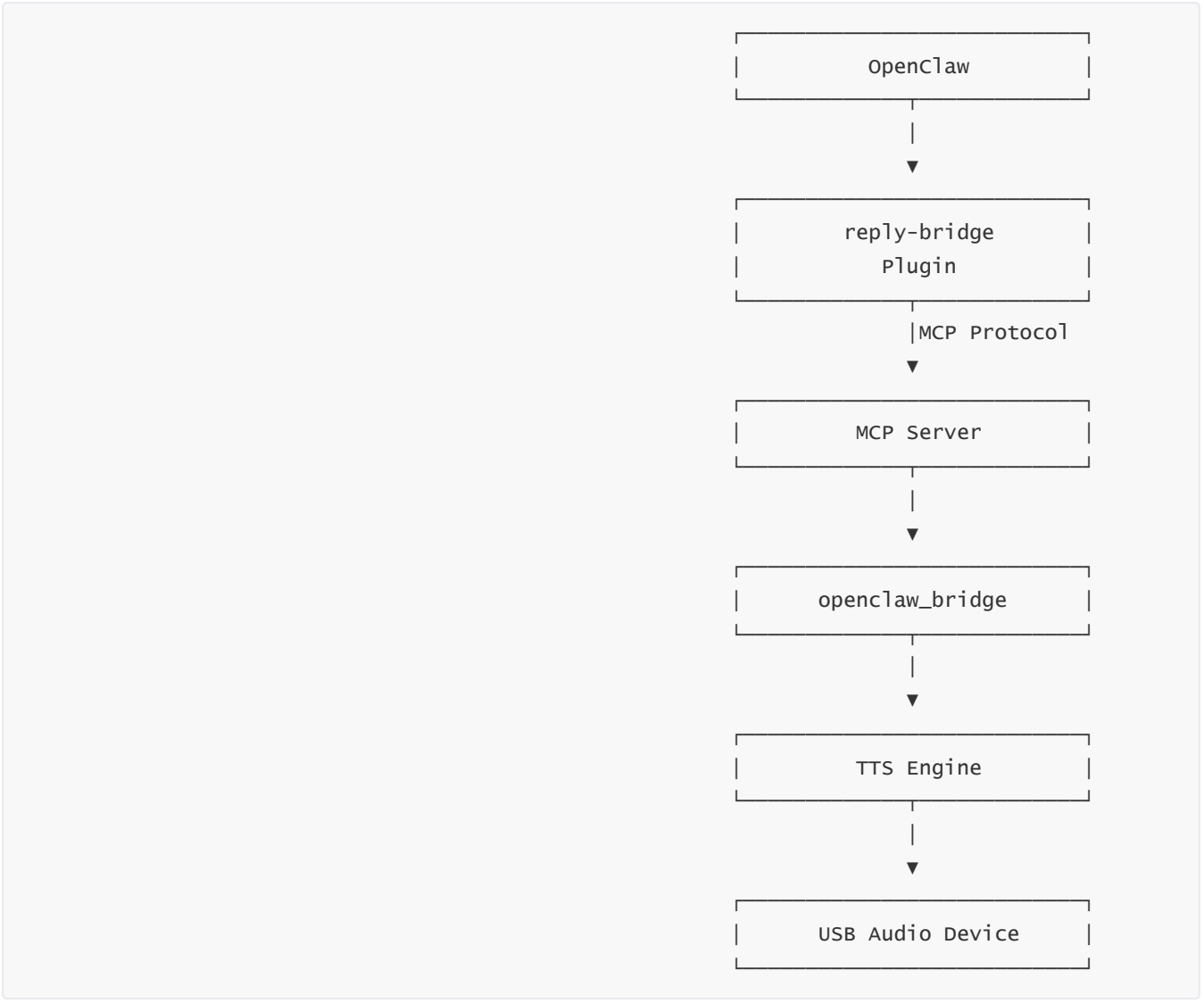
3. reply-bridge Plugin Introduction

The reply-bridge plugin is a speech synthesis bridge plugin adapted for the OpenClaw robot side. Its working principle is as follows:

- **Listening mechanism**: The plugin registers two Hooks, `before_message_write` and `message_sending`, to listen for the `<tts>...</tts>` tag in AI reply messages
- **Text extraction**: When an AI reply contains `<tts>tag text</tts>`, the plugin automatically extracts the text content within the tag
- **Speech synthesis**: Through the built-in MCP client, it calls the `TTS` tool on the MCP server to convert the text into speech
- **Audio broadcast**: The MCP server calls the TTS engine through the `openclaw_bridge` node, and finally broadcasts the speech through the USB audio device

- **Deduplication mechanism**: The plugin has a built-in session-level deduplication mechanism, so the same TTS text is not broadcast repeatedly within a session, avoiding redundant speech

The plugin communication architecture is as follows:



4. Case Demonstration

- Refer to the prerequisite chapter 【OpenClaw Interaction Method】—【OpenClaw Voice Dialogue Interaction】 for the dialogue feature; this section only explains the implementation principle of the voice dialogue feature

5. Source Code Analysis

5.1 Plugin Entry and Hook Registration

Source code directory:

```
$HOME/.openclaw/extensions/reply-bridge/index.ts
```

The plugin source code is located in `index.ts` under the plugin directory. It uses the `registerReplyBridge` function as the entry point and registers event listening Hooks using the OpenClaw Plugin API. The core code is as follows:

```
export default function registerReplyBridge(api: OpenClawPluginApi) {
  // Hook: before_message_write — triggered whenever a message is written to the session
record
  api.on("before_message_write", (event, ctx) => {
    const message = event.message as AgentMessage;
    const sessionId = ctx.sessionId;

    // Only handle messages with the assistant role
    if (message.role !== "assistant") return;

    const text = extractTextFromAgentMessage(message);
    if (!text) return;

    const ttsTexts = extractTtsTags(text);
    if (ttsTexts.length === 0) return;

    api.logger.info(`[reply-bridge] extracted ${ttsTexts.length} TTS segment(s)`);
    sendTtsTextsFireAndForget(ttsTexts, sessionId, (msg) => api.logger.info(msg));
  });

  api.logger.info("reply-bridge: Plugin registered (hooks: before_message_write,
message_sending)");
}
```

Code Description:

1. **Registration entry**: `export default function registerReplyBridge(api)` is the plugin's default export function. OpenClaw calls it automatically when loading the plugin, passing in the `OpenClawPluginApi` instance
2. `api.on("before_message_write")` : Registers the pre-message-write Hook, triggered when the AI generates a reply and writes it to the session record; this is the main channel for TTS processing
3. **Role filtering**: `message.role !== "assistant"` ensures that only AI assistant reply messages are processed, ignoring user messages
4. **Extraction flow**: `extractTextFromAgentMessage` → `extractTtsTags` → `sendTtsTextsFireAndForget`, three steps completing the full pipeline from message text to TTS broadcast
5. **fire-and-forget mode**: TTS calls use fire-and-forget mode, not blocking the message write flow,

ensuring a smooth user experience

Key Function Description:

- `extractTextFromAgentMessage(message)`: Extracts the plain text content from the AgentMessage object, handling the multimodal content that the message may contain (text, images, etc.)
- `extractTtsTags(text)`: Parses all content within `<tts>...</tts>` tags from the text and returns a list of TTS text segments
- `sendTtsTextsFireAndForget(ttsTexts, sessionId, logger)`: Asynchronously sends the extracted TTS text to the MCP server for speech synthesis, using fire-and-forget mode so as not to block the main flow
- `ctx.sessionId`: The current session ID, used for session-level TTS deduplication, ensuring the same text is not broadcast repeatedly within the same session

5.2 TTS Tag Extraction and Deduplication Mechanism

The code for the plugin to extract the content of the `<tts>` tag from the AI reply text is as follows:

```
/**
 * Extract plain text segments wrapped in <tts>...</tts> tags
 * Uses indexOf instead of regular expressions to avoid pattern matching boundary issues
 */
function extractTtsTags(content: string): string[] {
  const matches: string[] = [];
  const openTag = "<tts>";
  const closeTag = "</tts>";
  let start = 0;

  while (true) {
    const openIdx = content.indexOf(openTag, start);
    if (openIdx === -1) break;
    const closeIdx = content.indexOf(closeTag, openIdx + openTag.length);
    if (closeIdx === -1) break;
    const text = content.slice(openIdx + openTag.length, closeIdx).trim();
    if (text) matches.push(text);
    start = closeIdx + closeTag.length;
  }

  return matches;
}
```

Code Description:

1. `extractTtsTags()`: Uses `indexOf` to scan the text segment by segment, extracting all content within `<tts>...</tts>` tags. `indexOf` is chosen over regular expressions to avoid boundary issues with special characters and nested pattern matching, improving parsing stability
2. **Deduplication mechanism**: Records the text already broadcast for each session through `sentTtsTexts` (`Map<string, Set<string>>`), so the same text is not broadcast repeatedly within a session, avoiding redundant speech

Code Detail Description:

- `indexOf(openTag, start)`: Searches for the `<tts>` opening tag starting from the `start` position, returning the tag's starting index, or -1 if not found
- `indexOf(closeTag, openIdx + openTag.length)`: Searches for the `</tts>` closing tag after the opening tag, ensuring correct matching of the nearest closing position
- `content.slice(openIdx + openTag.length, closeIdx).trim()`: Extracts the text content between the opening and closing tags and removes leading and trailing whitespace
- `start = closeIdx + closeTag.length`: Updates the search start position to after the closing tag, supporting multiple `<tts>` tags in the same text
- `sentTtsTexts`: A `Map<sessionId, Set<text>>` structure, keyed by session ID with the set of broadcast texts as the value, achieving session-level deduplication

5.3 TTS Server

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/multi_brains_pre/multi_brains_pre/openclaw_bridge.py
```

- `init_tts_engine`: Initialize the TTS speech synthesis engine
    - When `USE_OLINE_TTS` is `False`: Load the local `PiperTTS` engine and use offline speech synthesis
    - When `tts_supplier` is `aliyun`: Load `TongyiTTS` (Alibaba Cloud Tongyi), requiring configuration of the API Key, model, and voice timbre
    - When `tts_supplier` is `xunfei`: Load `XunfeiTTS` (iFlytek), requiring configuration of the key path and encryption key
- `_process_tts_queue`: TTS queue processing worker thread
    - Loops to check the stop flag, and gets the text to be synthesized from the queue (timeout 0.5s)
    - Calls `_handle_tts` to process each piece of text
- `_handle_tts`: Handle a single TTS request
    - Waits for the `play_audio` Action server to be ready
    - Calls `tts_engine.synthesize` to synthesize the text into an audio file
    - Sends the `PlayAudio` Action goal, handling goal acceptance/rejection through callbacks
    - Waits for playback to complete, checking the stop flag during this period, and supports canceling the ongoing playback

Code Detail Description:

- `self.USE_OLINE_TTS`: Offline TTS switch; when `False`, it loads the local PiperTTS engine, and when `True`, it loads the online engine according to `tts_supplier`
- `self.tts_supplier`: Online TTS provider selection, supporting `aliyun` (Alibaba Cloud Tongyi) and `xunfei` (iFlytek)
- `self.tts_engine.synthesize(text, self.tts_out_path)`: Calls the TTS engine to synthesize the text into an audio file, saving it to the `tts_out_path` path

- `self.audio_client.wait_for_server(timeout_sec=3.0)`: Waits for the `play_audio` Action server to come online; if it times out after 3 seconds, it skips this broadcast
- `PlayAudio.Goal()`: Constructs the Action goal message for playing audio, with `file_path` specifying the synthesized audio file path
- `_goal_response_cb`: Action goal send callback; checks whether the server accepts the goal, and if rejected, logs a warning and ends
- `_tts_goal_handle_lock`: Thread lock protecting concurrent access to `_current_tts_goal_handle`, ensuring the safety of cancellation operations
- `_result_cb`: Action execution completion callback; clears the current goal handle and sets `done_event` to notify the main thread
- `done_event.wait(timeout=0.1)`: The main loop polls the playback completion status at 0.1-second intervals, checking the stop flag during this period to support canceling playback
- `gh.cancel_goal_async()`: When the stop flag is detected, asynchronously cancels the currently playing Action goal

```
    def init_tts_engine(self):
        if not self.USE_OLINE_TTS:
            from multi_brains_pre.utils.tts import  PiperTTS
            self.tts_engine=PiperTTS(self.language)
            if self.tts_engine.init_tts_engine() :
                self.get_logger().info(Fore.GREEN+"Piper TTS_Engine initialized
successfully"+Fore.RESET)
            else:
                error_msg="Piper TTS_Engine initialized failed"

                self.get_logger().error(Fore.RED+error_msg+Fore.RESET)
                raise RuntimeError(error_msg)
            return

        if self.tts_supplier=="aliyun":
            from multi_brains_pre.utils.tts import TongyiTTS

 self.tts_engine=TongyiTTS(self.aliyun_api_key,self.tts_model,self.tts_voice_tone)
            result=self.tts_engine.init_tts_engine()
            if result[0]:
                self.get_logger().info(Fore.GREEN+"Tongyi TTS_Engine initialized
successfully"+Fore.RESET)
            else:
                self.get_logger().error(Fore.RED+f"Tongyi TTS_Engine initialized failed:
{result[1]}"+Fore.RESET)

                raise RuntimeError(f"Tongyi TTS_Engine initialized failed:{result[1]}")


        elif self.tts_supplier=="xunfei":
            from multi_brains_pre.utils.tts import  XunfeiTTS

 self.tts_engine=XunfeiTTS(self.key_path,self.xunfei_encrypted_key,self.config_file)
```

```
            if self.tts_engine.init_tts_engine():
                self.get_logger().info(Fore.GREEN+"Xunfei TTS_Engine initialized
successfully"+Fore.RESET)
            else:

                error_msg="Xunfei TTS_Engine initialized failed"
                self.get_logger().error(Fore.RED+error_msg+Fore.RESET)
                raise RuntimeError(error_msg)
    def _process_tts_queue(self):
        """TTS """
        while not self._tts_worker_stop_event.is_set():
            try:
                text = self._tts_queue.get(timeout=0.5)
            except queue.Empty:
                continue
            self._handle_tts(text)

    def _handle_tts(self, text: str):
        if not self.audio_client.wait_for_server(timeout_sec=3.0):
            self.get_logger().warn(Fore.YELLOW + "play_audio action server not available,
skip voice" + Fore.RESET)
            return
        if not self.tts_engine.synthesize(text, self.tts_out_path):
            self.get_logger().error(Fore.RED + "Speech synthesis failed. Check whether the
TTS model is available" + Fore.RESET)
            return

        goal = PlayAudio.Goal()
        goal.file_path = self.tts_out_path

        done_event = threading.Event()

        def _goal_response_cb(future):
            goal_handle = future.result()
            if not goal_handle or not goal_handle.accepted:
                self.get_logger().warn("TTS play goal rejected")
                done_event.set()
                return
            with self._tts_goal_handle_lock:
                if self._tts_worker_stop_event.is_set():
                    goal_handle.cancel_goal_async()
                    done_event.set()
                    return
                self._current_tts_goal_handle = goal_handle
            result_future = goal_handle.get_result_async()
            result_future.add_done_callback(_result_cb)

        def _result_cb(future):
            with self._tts_goal_handle_lock:
                self._current_tts_goal_handle = None
            done_event.set()
```

```
        send_future = self.audio_client.send_goal_async(goal)
        send_future.add_done_callback(_goal_response_cb)

        while not done_event.wait(timeout=0.1):
            if self._tts_worker_stop_event.is_set():
                with self._tts_goal_handle_lock:
                    gh = self._current_tts_goal_handle
                if gh is not None:
                    gh.cancel_goal_async()
                break
```